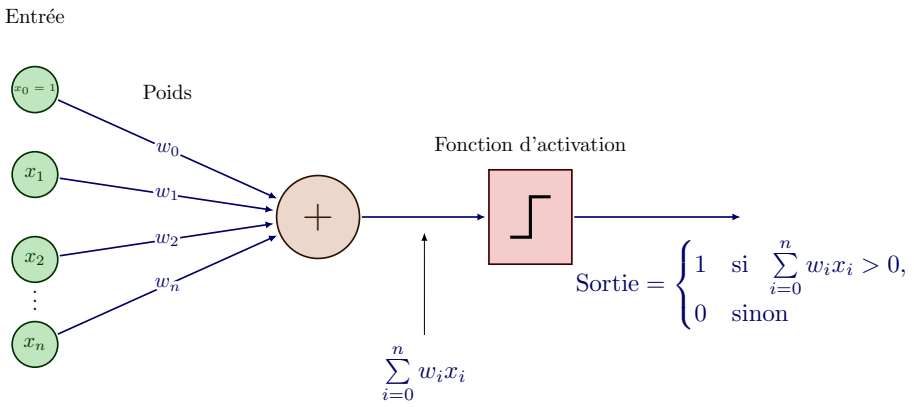
IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

X62 - Optimisation pour l'aide à la décision
TP n° 3
Perceptron

La définition du perceptron et le processus d'apprentissage associé ont été vus en M61.

Pour mémoire le schéma suivant résume son fonctionnement et rappelle les notations d'usage (formalisme projectif).



L'objectif du TP est d'implémenter en python une classe `Perceptron`, puis de tester et de visualiser son algorithme d'apprentissage sur des jeux de donnés variés.

1 La classe Perceptron

Paramètres et initialisation

Définir une classe Perceptron avec un constructeur prenant en paramètre la dimension des données (vecteur d'entrée) du perceptron, et créant et initialisant ses caractéristiques :

```
class Perceptron:
    def __init__(self,  dim=2):
  # création et initialisation des paramètres  caractéristiques (poids, biais)
```

Notez que plusieurs formalismes sont possibles : poids et seuil, poids et biais, modèle homogène (poids uniquement). Vous êtes libre d'adopter celui de votre choix.

De même l'initialisation peut se faire de différentes manières.

Méthode calculant la sortie du perceptron

Il est maintenant possible de définir la fonction calculant la sortie du perceptron pour une donnée $X = (x_1, \ldots, x_d)$.

```
    def output(self, X):
       # retourne la sortie du perceptron pour une donnée X en entrée
```

Méthode d'apprentissage du perceptron

Le cœur du TP est l'implémentation de la méthode d'apprentissage du perceptron sur un jeu de données `(X,y)`

```
    def train(self, X, y, nb_epoch=100):
        # Pour chaque epoch (itération sur l'ensemble des données)
        #    Pour chaque élément x du jeu de données
        #            calculer la sortie du perceptron pour x
        #            calculer l'erreur pour cet élément
        #            mettre à jour les paramètres en conséquence
```

2 Premiers tests et visualisation de l'apprentissage en dimension 2

Pour tester le bon fonctionnement de l'implémentation, on va commencer par des jeux de données simples en dimension 2 :

1. le graphe de la fonction booléenne OR

```
X_or = np.array([[0,0], [1,1], [1,0], [0,1]])
y_or = np.array([0, 1, 1, 1])
```

2. puis celui de la fonction XOR.

On rappelle que le premier exemple constitue un ensemble de données séparable, pour lequel l'algorithme d'apprentissage converge, tandis que le second n'est pas séparable.

Pour évaluer la qualité et la vitesse de la convergence, on visualisera l'évolution de l'erreur globale, c'est-à-dire la somme des erreurs pour chaque élément du jeu de données lors d'une itération (notez qu'il y a deux interprétations possibles de cette définition de la *loss*, selon qu'on corrige les poids au fur et à mesure de la traversée de l'époque, ou non, mais que ça n'a pas d'importance du point de vue de l'observation de la convergence) en fonction du nombre d'*époques*, ainsi que le résultat de la classification binaire une fois l'entraînement du perceptron terminé.

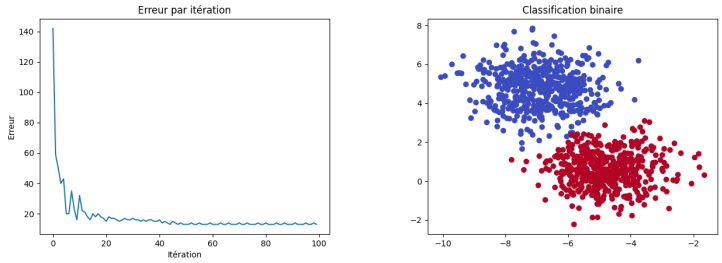


FIGURE 1 – Exemple de visualisation de l'évolution de l'erreur globale et de la classification binaire finale sur des données générées

Pour générer facilement et rapidement des exemples plus complexes, on propose d'utiliser la fonction `make_blobs` fournie par la bibliothèque `scikit-learn`

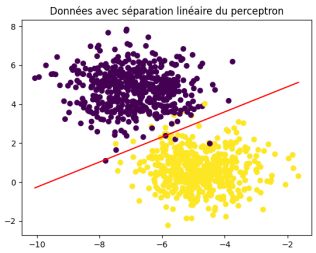
```
from sklearn.datasets import make_blobs
X, y = make_blobs(n_features=2, centers=2, n_samples=1000, random_state=12)
```

On testera alors l'entraînement de la classe sur trois ensembles de données de taille 1000 :

1. un ensemble de données séparable : l'algorithme converge et réussit la classification ;
2. un ensemble de données presque séparable (comme l'exemple donné ci-dessus) : l'algorithme ne converge pas mais donne une classification approximative raisonnable ;
3. un ensemble de données non séparable.

Notez bien que la deuxième catégorie ne relève que d'une commodité de langage, et non d'une définition mathématique.

Pour mieux apprécier la pertinence de la classification fournie, on visualisera la droite de séparation obtenue, simultanément avec les données d'entraînement :



3 Passage aux dimensions supérieures

Le principe et l'algorithme d'apprentissage du perceptron s'étendent sans modification au cas des dimensions supérieures à 2, mais il est clair que la visualisation et l'évaluation des résultats doivent être adaptées.

En dimension 3

Pour des données de dimension 3, on peut encore évaluer de manière assez directe la pertinence de la classification selon la séparabilité des données en entrée.

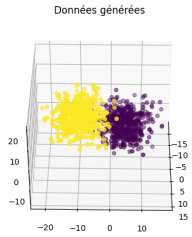
On conseille de commencer par traiter un cas simple et prévisible de données construites « à la main », par exemple

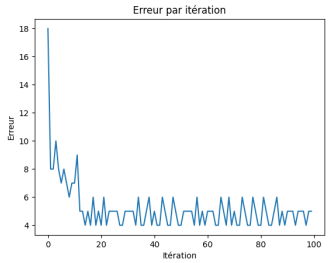
```
X_c = np.array([[0,0,0], [0,0,1], [0,1,0], [0,1,1], [1,0,0], [1,1,0], [1,0,1], [1,1,1]])
y_c = np.array([0, 0, 0, 0, 0, 0, 1, 1])
```

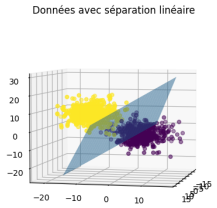
On demande ensuite pour chacun des cas énumérés à la section précédente (séparabilité, quasi-séparabilité et non séparabilité)

- de générer des jeux de données au moyen de `make_blobs`;
- d'entraîner un perceptron sur ces données;
- puis de visualiser le résultat de l'apprentissage de la même façon qu'en deux dimensions.

Ici, le séparateur linéaire obtenu en fin d'apprentissage est un plan, et on obtiendra une synthèse graphique des résultats du type





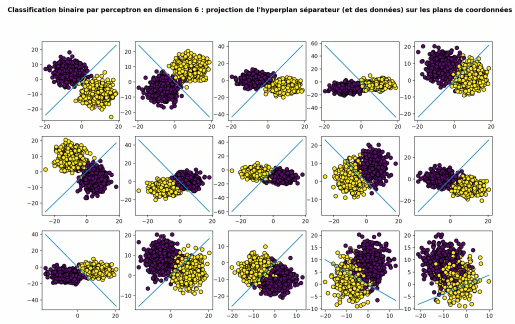


En dimension supérieure

Pour tester le perceptron en dimension plus grande que 3, on pourra toujours s'appuyer sur la génération de données par `make_blobs`, et sur l'évolution de la fonction de coût au fil des itérations.

En revanche la visualisation des données et de la classification (le séparateur linéaire est à présent un hyperplan) ne peut pas s'effectuer directement. On propose donc de visualiser ces éléments au moyen de projection sur les différents plans engendrés par les axes de coordonnées.

Voici un exemple en dimension 6 :



Il faut être conscient que la séparabilité des données ne se lit pas sur ces graphiques (comment la contrôle-t-on de manière sûre ?), pas davantage que la correction de l'hyperplan séparateur obtenu, mais on peut néanmoins se convaincre que le résultat n'est pas aléatoire !

Remarque. La projection sur les plans de coordonnées est une approche naïve, mais qui a l'avantage d'être élémentaire à la mise en œuvre. Il existe d'autres techniques, plus élaborées, qui consistent à choisir des plans (ou des sous-espaces) de projection particulièrement adaptés aux données étudiées.